

SEMINÁRIOS

Explorando Ferramentas de Build

1. Introdução

Ferramentas de build são programas utilizados no desenvolvimento de software para automatizar o processo de compilação e montagem de código-fonte em um aplicativo executável ou em outro formato distribuível. Essas ferramentas geralmente automatizam tarefas como compilação de código, gerenciamento de dependências, execução de testes e criação de artefatos de distribuição. A lista abaixo apresenta alguns exemplos de ferramentas de build disponíveis:

- **Apache Maven:** Uma ferramenta de automação de compilação e gerenciamento de projetos Java. Maven utiliza um formato de construção baseado em XML e gerencia dependências automaticamente.
- **Gradle:** Uma ferramenta de automação de compilação que suporta vários idiomas, incluindo Java, Groovy e Kotlin. Ele oferece flexibilidade e desempenho, permitindo que os desenvolvedores escrevam scripts de build usando Groovy ou Kotlin DSL.
- **Ant:** Outra ferramenta de automação de compilação Java, semelhante ao Maven. Ant usa scripts XML para definir o processo de compilação e é altamente configurável.
- **Make:** Uma das ferramentas de build mais antigas e amplamente utilizadas, originalmente desenvolvida para compilar programas em C, mas agora é utilizada em muitos outros contextos. Os arquivos Makefile definem as regras de compilação e as dependências entre os componentes do software.

- **Webpack:** Uma ferramenta de build moderna e popular para aplicativos da web. O Webpack é usado principalmente para empacotar recursos da web, como JavaScript, CSS e imagens, em pacotes otimizados para produção.
- **Bazel:** Uma ferramenta de build de código aberto desenvolvida pelo Google e usada para criar muitos de seus projetos internos. Bazel é escalável e suporta várias linguagens de programação.
- **CMake:** Uma ferramenta de build projetada para ser independente de plataforma e de compilador, frequentemente usada para construir projetos em C++ e C.
- **Grunt:** Uma ferramenta JavaScript de automação de tarefas que pode ser usada para várias tarefas de build, como minificação de arquivos, concatenação e execução de testes.
- **Gulp:** Outra ferramenta JavaScript para automação de tarefas, semelhante ao Grunt, mas com uma abordagem mais baseada em código. Gulp permite que os desenvolvedores escrevam tarefas como streams de código.
- **Rake:** Uma ferramenta de build escrita em Ruby, frequentemente usada em projetos Ruby on Rails. Rake usa scripts Ruby para definir as tarefas de build e suas dependências.

1.1. Características

1. **Geração do script de construção:** O sistema de build deve analisar o programa em construção, identificar componentes dependentes e gerar automaticamente um script de construção (arquivo de configuração). Além disso, deve oferecer a capacidade de criação e edição manual de scripts de construção, proporcionando flexibilidade ao desenvolvedor.
2. **Integração com o sistema de controle de versão:** O sistema de construção (build) deve sincronizar-se com o sistema de controle de versão para realizar o check-out das versões necessárias dos componentes. Isso garante a consistência e rastreabilidade das dependências do projeto.

3. **Recompilação mínima:** O sistema de build deve determinar quais partes do código-fonte precisam ser re-compiladas e realizar apenas as compilações necessárias. Essa abordagem otimiza o tempo de construção, garantindo eficiência no processo de desenvolvimento.
4. **Criação do sistema executável:** O sistema de build deve conectar os arquivos objeto compilados entre si e com outros arquivos necessários, como bibliotecas e arquivos de configuração, para gerar um sistema executável funcional. Isso é essencial para garantir a correta execução do software desenvolvido.
5. **Automação dos testes:** Alguns sistemas de build são capazes de automatizar a execução de testes utilizando ferramentas de automação, como JUnit. Esses testes são cruciais para verificar se as mudanças introduzidas no código não quebram a integridade do sistema.
6. **Emissão de relatórios:** O sistema de build deve fornecer relatórios claros e detalhados sobre o sucesso ou falha da construção, bem como os resultados dos testes executados. Essa funcionalidade é fundamental para o monitoramento da qualidade do software em desenvolvimento.
7. **Geração da documentação:** O sistema de build pode incluir recursos para gerar notas de lançamento sobre o build realizado e criar páginas de documentação do sistema. Isso auxilia na comunicação eficaz com os usuários finais e na manutenção do software ao longo do tempo

2. Objetivo

O objetivo deste seminário é explorar e apresentar diversas ferramentas de build utilizadas no desenvolvimento de software, destacando suas características, vantagens e desvantagens. Cada grupo deverá escolher uma ferramenta de build para pesquisar e apresentar. As opções de ferramentas são diversas e não se limitam aos exemplos fornecidos na introdução deste documento.

Além disso, para evitar duplicidade de apresentações, não serão permitidas repetições de ferramentas durante as sessões. Portanto, após a escolha da ferramenta, um representante de cada grupo deverá postar o nome e o link do website oficial da

ferramenta selecionada no fórum do Canvas. O primeiro grupo que postar no fórum apresentará a ferramenta escolhida.

3. Tema

Com a ferramenta escolhida, pesquise os aspectos cruciais da ferramenta de build escolhida, tais como:

- **Facilidade de Configuração:** Como a ferramenta facilita a configuração e personalização do processo de build?
- **Eficiência de Compilação:** Como a ferramenta otimiza o processo de compilação, minimizando o tempo necessário para gerar o artefato final?
- **Gerenciamento de Dependências:** De que maneira a ferramenta lida com as dependências do projeto, garantindo sua integridade e disponibilidade?
- **Suporte a Multiplataformas:** Como a ferramenta suporta diferentes plataformas e sistemas operacionais?
- **Automação de Tarefas:** Como a ferramenta automatiza tarefas repetitivas relacionadas ao build, como compilação, teste e empacotamento?
- **Extensibilidade:** Em que medida a ferramenta permite a extensão de suas funcionalidades por meio de plugins ou outras formas de customização?
- **Comunidade e Suporte:** Qual é o suporte oferecido pela comunidade em torno da ferramenta, incluindo documentação, fóruns de discussão e atualizações regulares?

Além disso, faça uma demonstração prática ao vivo em sala de aula do uso da ferramenta, mostrando como configurá-la, realizar o build de um projeto simples e examinar o artefato resultante. E apresente as vantagens e desvantagens da ferramenta, destacando pontos positivos para auxiliar na tomada de decisão sobre sua utilização em projetos reais.

4. Apresentação

Os grupos apresentarão seus seminários para a turma, compartilhando suas descobertas sobre a ferramenta de build escolhida. Cada apresentação terá duração de 10 minutos e durante as apresentações, os grupos devem garantir que o tempo seja gerenciado de forma eficaz, permitindo espaço para explicar cada ponto de maneira clara e concisa, enquanto ainda reservam tempo para a demonstração prática ao vivo e para responder a possíveis perguntas da plateia.

5. Avaliação

- Os grupos serão avaliados com base na compreensão das características da ferramenta de build escolhida, na qualidade do exemplo prático desenvolvido e na capacidade de comunicação durante a apresentação.
- Os demais alunos serão incentivados a participar com perguntas e comentários construtivos para enriquecer a discussão